

PARALLEL EVOLUTIONARY ALGORITHMS

João Luís Sobral, Jorge Pinho, Tiago Martins

Departamento de Informática

Universidade do Minho

Braga, Portugal



Evolutionary Algorithms overview

2/19

□ Evolutionary Algorithms (EAs)

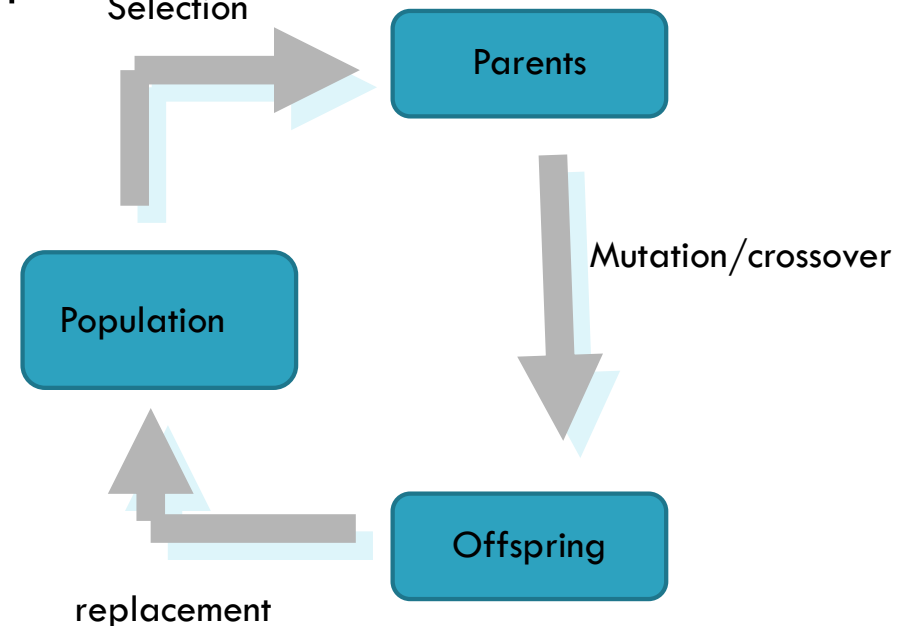
- ▣ Used to solve many complex optimization problems

- ▣ Inspired in Darwin's theory of natural evolution

 - Selection, mutation/crossover, replacement

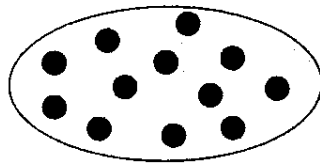
▣ Main algorithm

```
t := 0;  
initialize and evaluate [P(t)];  
while not stop_condition do  
  P'(t) := variation [P(t)];  
  evaluate [P'(t)];  
  P(t+1) := select [P'(t), P(t)];  
  t := t + 1;  
end while;
```

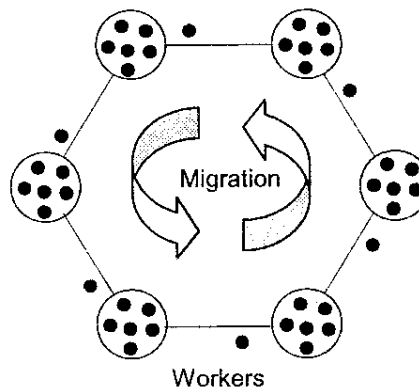


Parallel EAs

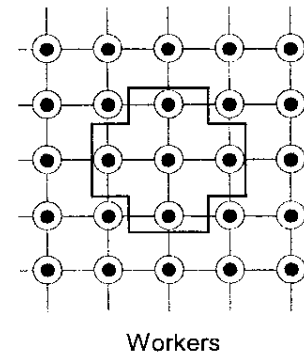
3/19



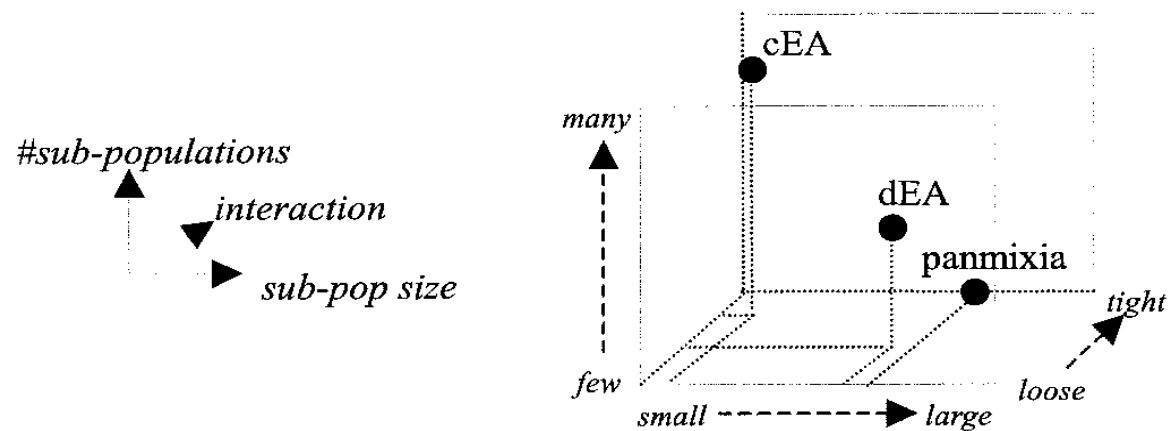
panmixia



dEA (coarse grain)



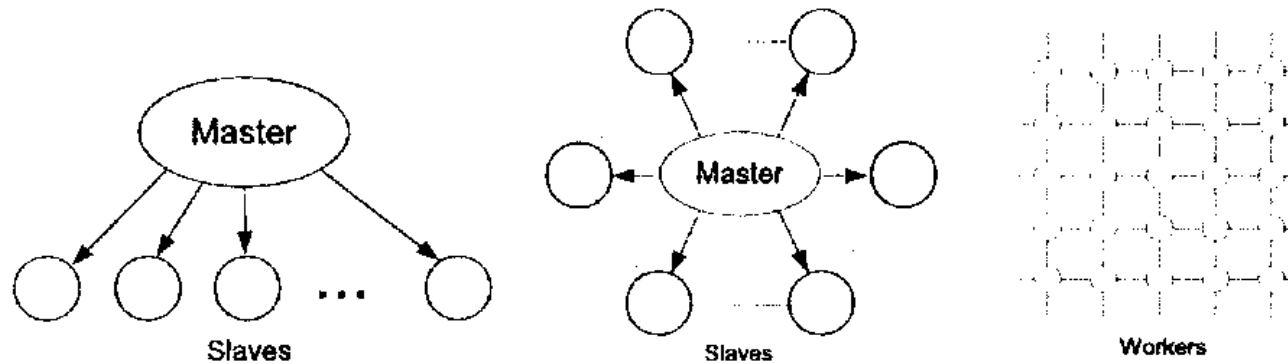
cEA (fine grain)



Parallel EAs

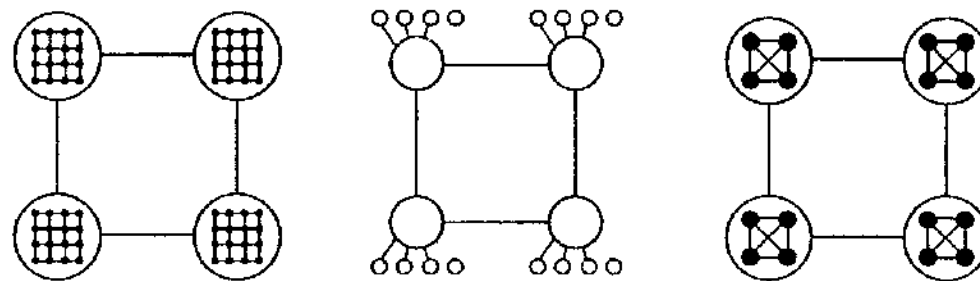
4/19

□ Base parallelism models



(a) global parallelization; (b) coarse grain; and (c) fine grain.

□ Hybrid models



(d) coarse and fine grain; (e) coarse grain and global parallelization; and (f) coarse grain at the two levels.

Parallel EAs

5/19

- **Measuring speed-up**
 - **EAs are stochastic algorithms**
 - Use the average of a number of runs
 - **Strong definition of speed-up**
 - Use as T_s the best sequential algorithm
 - Not always possible
 - **Weak definition of speed-up**
 - Comparing own sequential and parallel algorithms.
 - **Stopping criterion**
 - **Number of iterations:** breaks the fundamental statement of being “solving” the same problem with the “same” precision
 - ***orthodox weak*** : stop when a solution of the same quality had been found

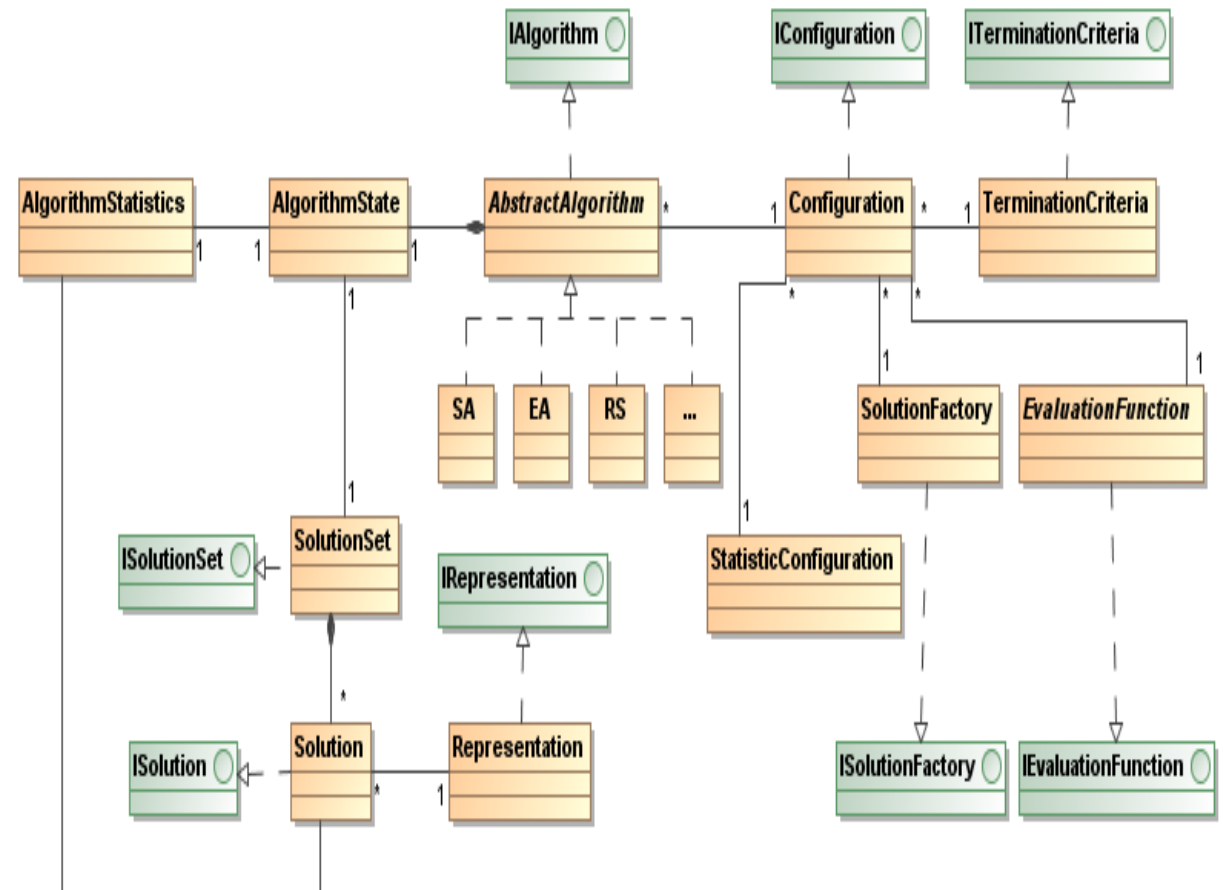
JECoLi overview

6/19

Java Evolutionary Computation Library (JEColi)

■ Main abstractions

- Solutions
 - Encoding
- Evaluation Function
- Termination Criteria
- Algorithm
 - Optimization method
- Configuration
 - Algorithm configuration



CountOnesEATest

7/19

```
public static void main(String[] args) {

    EvolutionaryConfiguration<ILinearRepresentation<Boolean>> configuration = new
EvolutionaryConfiguration<ILinearRepresentation<Boolean>>();

    IEvaluationFunction<ILinearRepresentation<Boolean>> evaluationFunction = new CountOnesEvaluationFunction();
    configuration.setEvaluationFunction(evaluationFunction);

    int populationSize = 10;
    configuration.setPopulationSize(populationSize);

    int numberGenerations = 2000;
    ITerminationCriteria terminationCriteria = new IterationTerminationCriteria(numberGenerations);
    configuration.setTerminationCriteria(terminationCriteria);

    RecombinationParameters recombinationParameters = new RecombinationParameters(populationSize);
    configuration.setRecombinationParameters(recombinationParameters);

    configuration.setSelectionOperators(new TournamentSelection<ILinearRepresentation<Boolean>>(1, 4));

    ReproductionOperatorContainer operatorContainer = new ReproductionOperatorContainer();
    operatorContainer.addOperator(0.5, new UniformCrossover<Boolean>());
    operatorContainer.addOperator(0.5, new BitFlipMutation(5));
    configuration.setReproductionOperatorContainer(operatorContainer);

    IAlgorithm<ILinearRepresentation<Boolean>> algorithm = new EvolutionaryAlgorithm<ILinearRepresentation<Boolean>>(configuration);

    IAlgorithmResult<ILinearRepresentation<Boolean>> results = algorithm.run();

}
```

Objectives

8/19

□ JECoLi Parallelization

- ▣ Original version was sequential
 - Could not benefit from multi-core, cluster and grid.
- ▣ Well known techniques to develop parallel EA
 - Parallel evaluation
 - Island model (with migration)
 - Hybrid (multi-island, island with parallel evaluation, ...)
- ▣ Our goal:
 - Pluggable parallelization: modular, non-invasive and developed independently
 - Support multiple kinds of parallelization models / target platforms
 - Easy to use and understand

Features

9/19

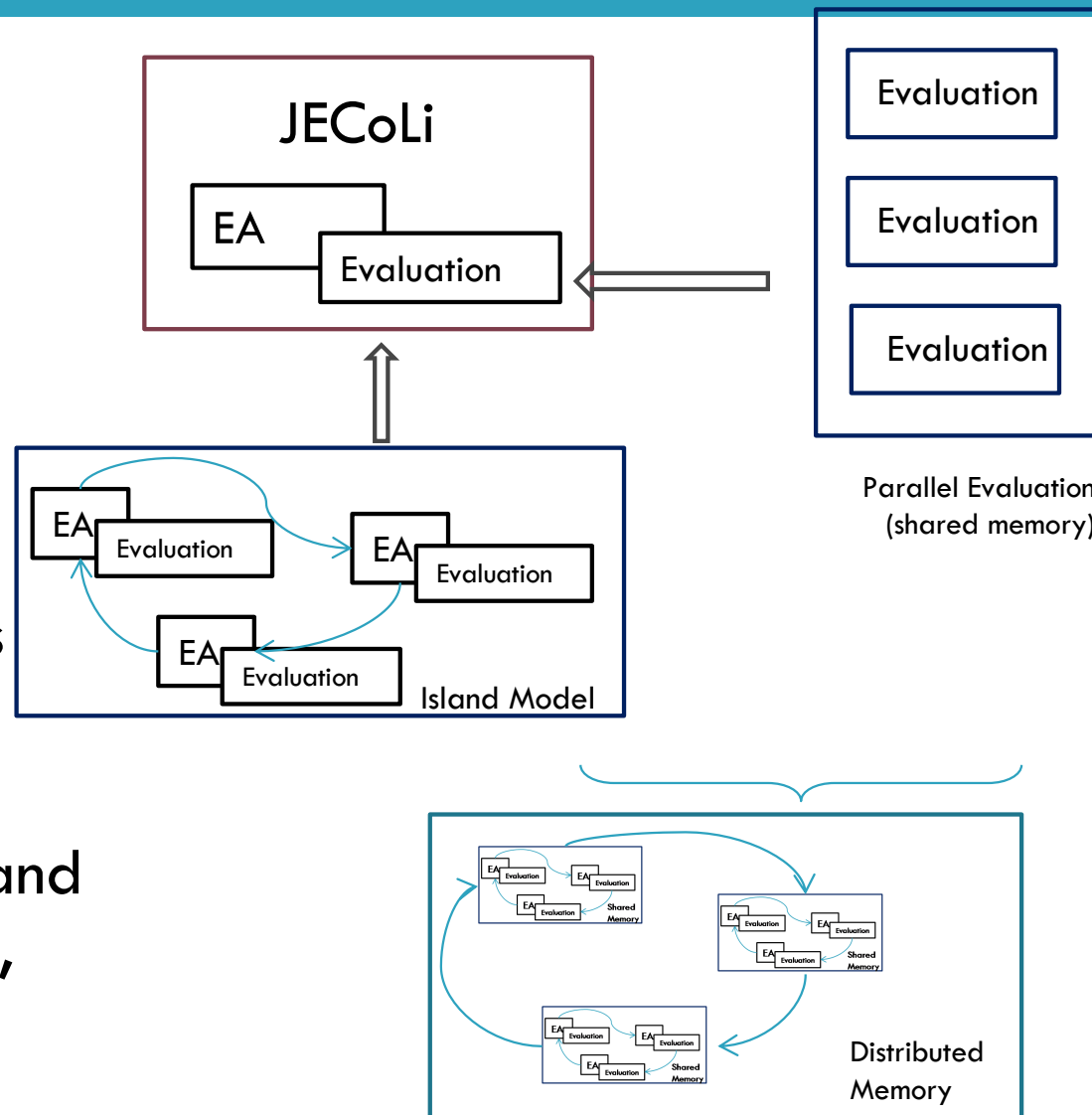
- What is parallelJEColi
 - ▣ Parallelism features
 - Island Model
 - Parallel Evaluation
 - ▣ Platform Mappings
 - Multicore
 - Cluster
 - Grid
 - ▣ Composition tool
 - Customization windows (GUI)

Parallelism features

10/19

□ Parallelization models

- ▣ **Parallel evaluation** – evaluate solutions in parallel within one generation
- ▣ **Island model** (with migration) – create multiple sets of solutions that evolve in a loosely coupled manner
- ▣ **Hybrid** (multi-island, island with parallel evaluation, ...)



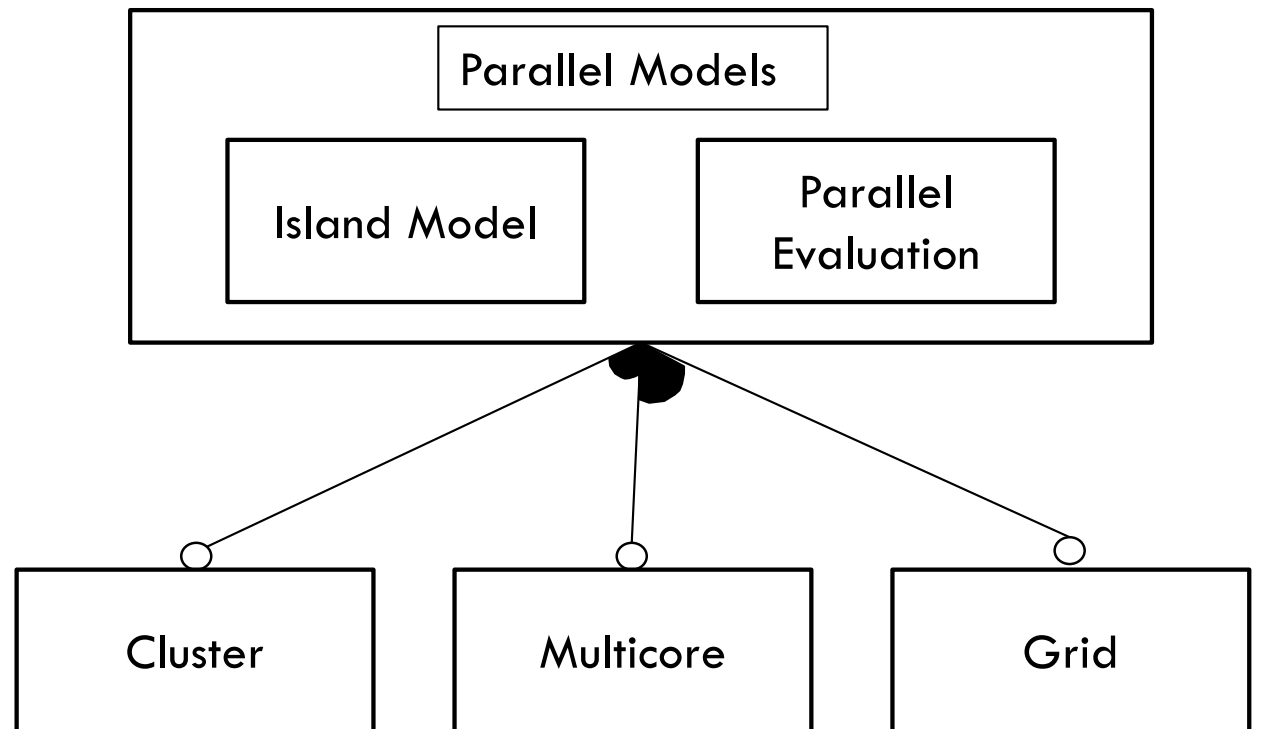
Platform mappings

11/19

□ Supported architectures

- ▣ Multicore
- ▣ Cluster
- ▣ Grid

□ mapping of the parallelization features to each platform



ParJEColi Layered architecture

12/19

□ 3rd Layer

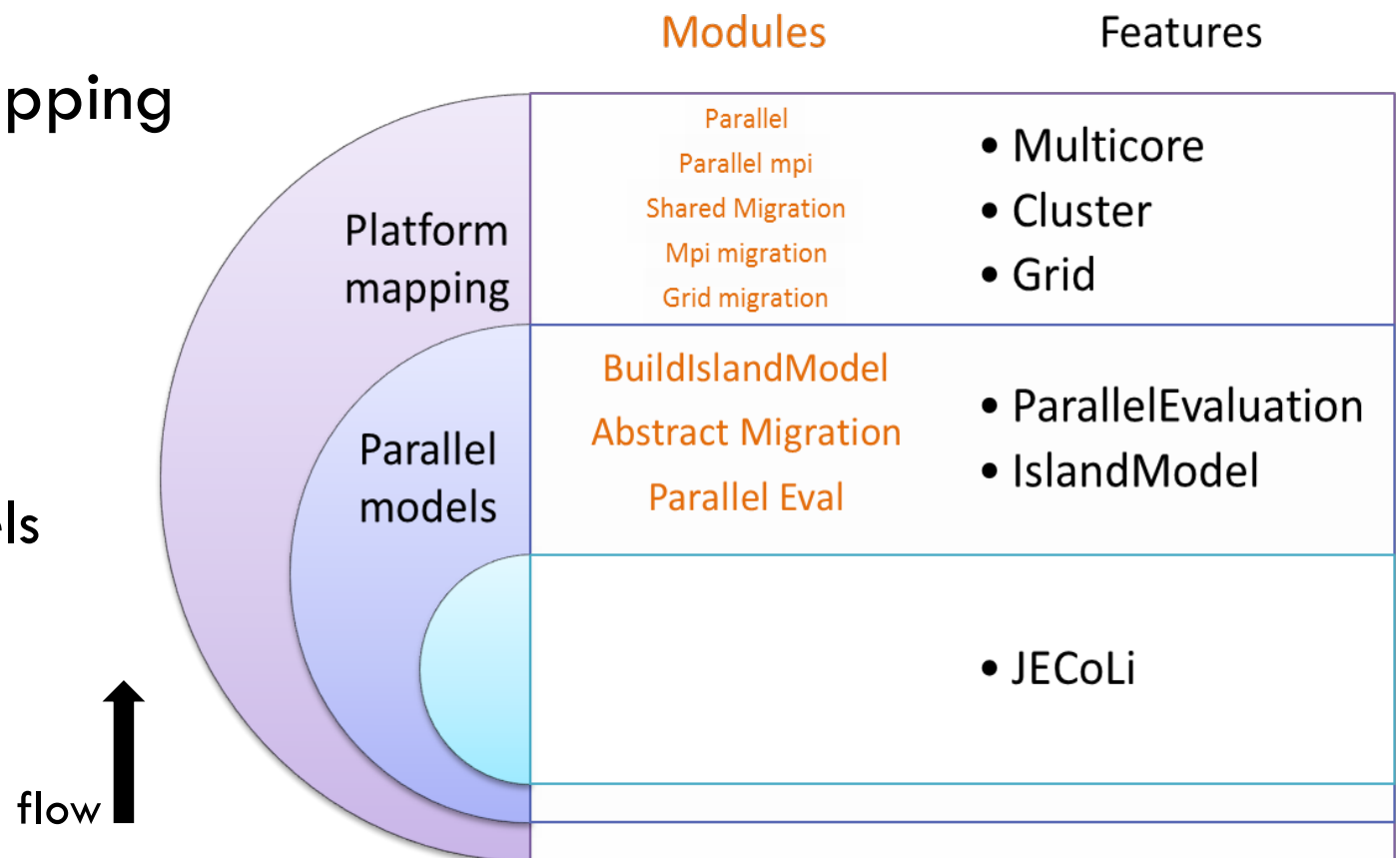
▣ Platform mapping

□ 2nd Layer

▣ Parallel models

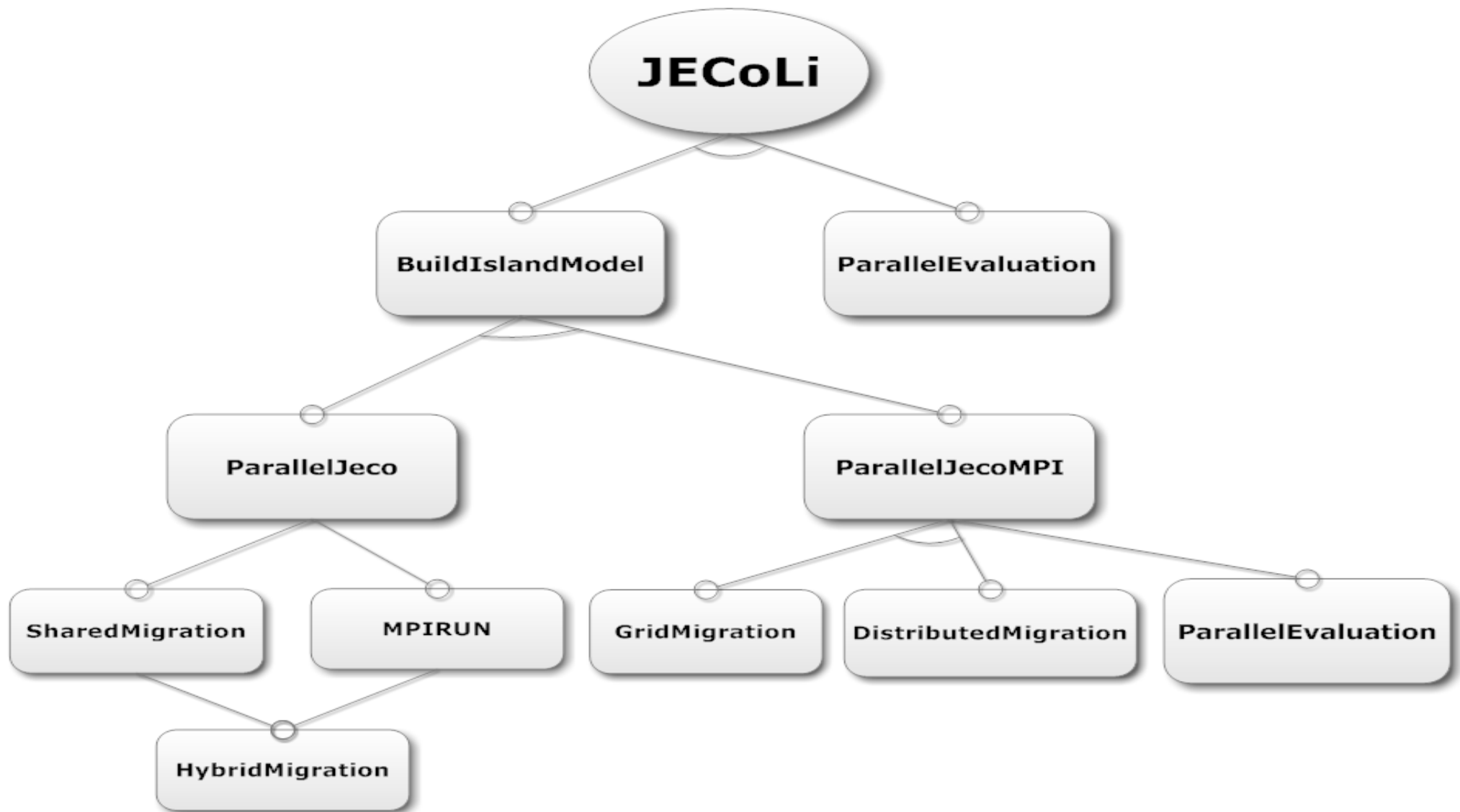
□ 1st Layer

▣ Optimization method (JEColi)



Composition tool : feature model

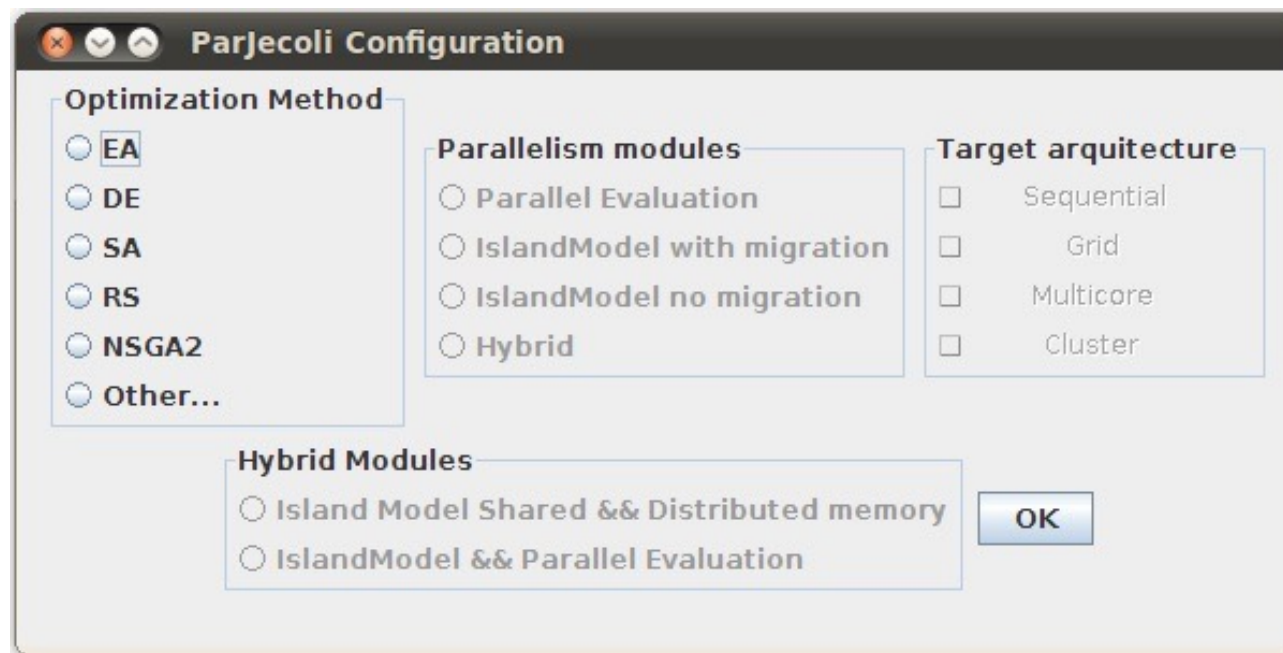
13/19



Composition tool – customization window

14/19

- ❑ 1st - Choose Optimization method
- ❑ 2nd - Choose Parallel module
- ❑ 3rd - Choose target architecture
- ❑ “OK” – Write xml file with specified configuration
 - ▣ Load Time weaving (`-javaagent:aspectjweaver.jar`)



Case study: JEColi Parallelisation

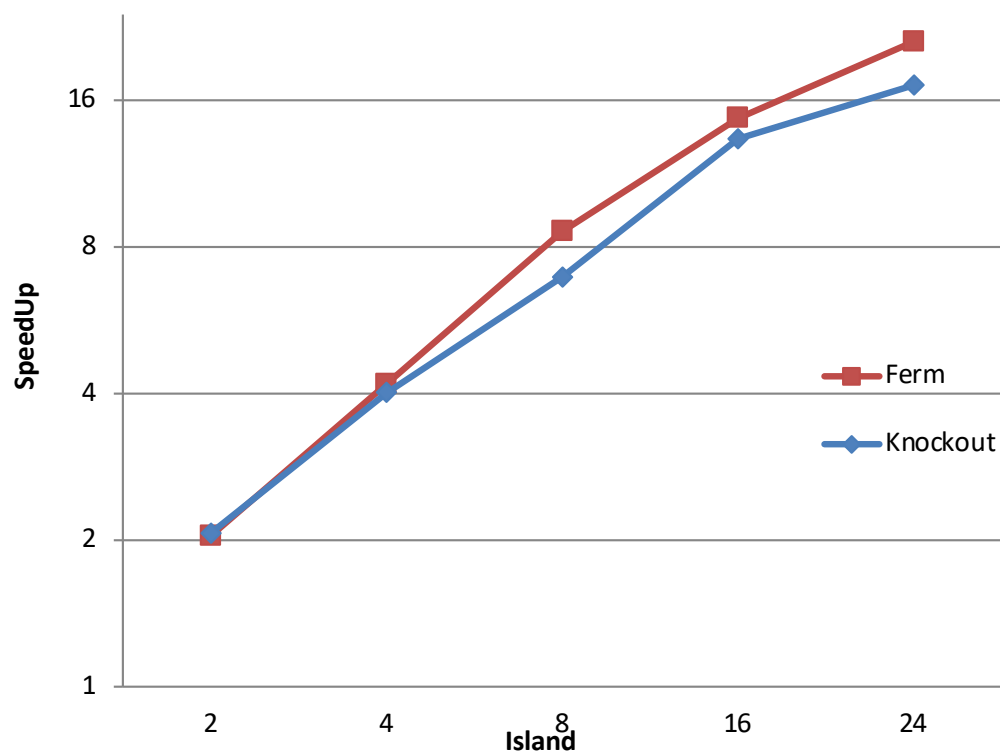
15/19

□ Benchmark

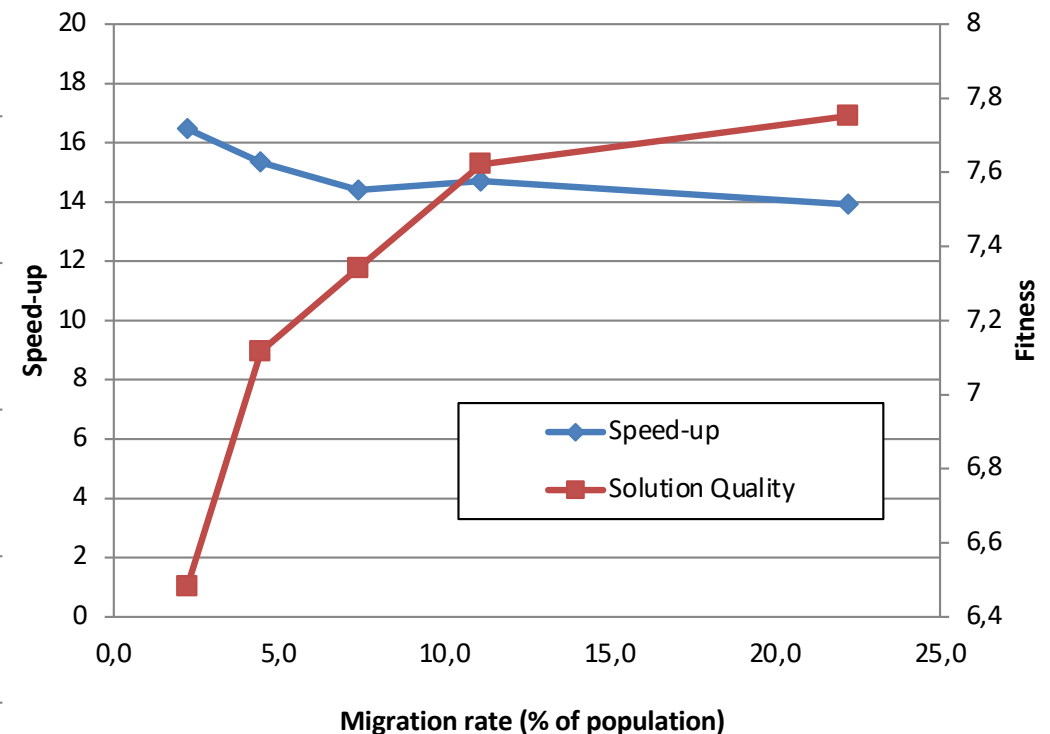
- Optimization of fed-batch **fermentations** to achieve optimal feed-forward control
 - find the optimal input feeding trajectories, in order to improve the process performance
- Optimization *in silico* of Metabolic Engineering (**knockout**)
 - selects, from a set of genes in a microbe's genome, a subset to be deleted in order to maximize a given objective function
- **Cluster of 4 machines**
 - Dual - Xeon E5520 (i7-Quad@2.26 GHz w/HT), 8 GB RAM
 - Myrinet 10Gb/s
 - mpiJava over OpenMPI
- Grid: 4 x Intel(R) Xeon(R) CPU E5345@2.33GHz, 768MB
- Intel i7 (quad core) @ 2.67GHz with Hyper Threading enabled, 6 GB RAM.

Results (old-Jorge Pinho)

16/19

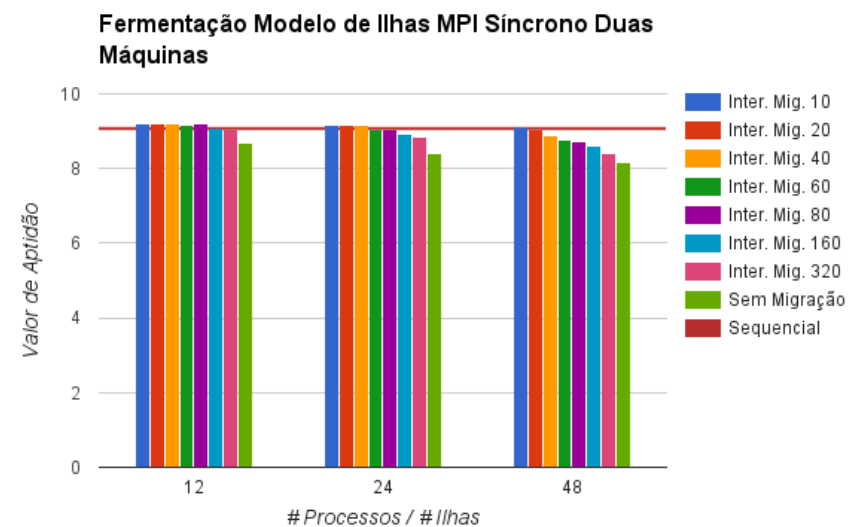
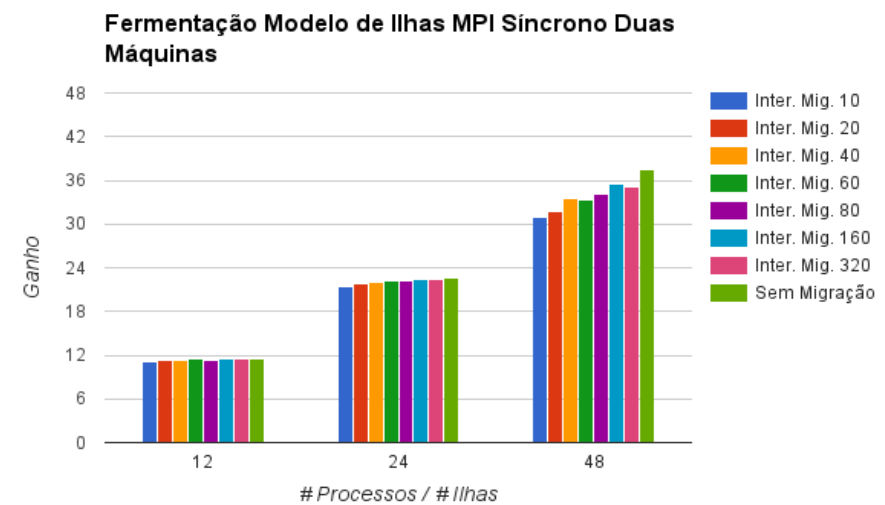
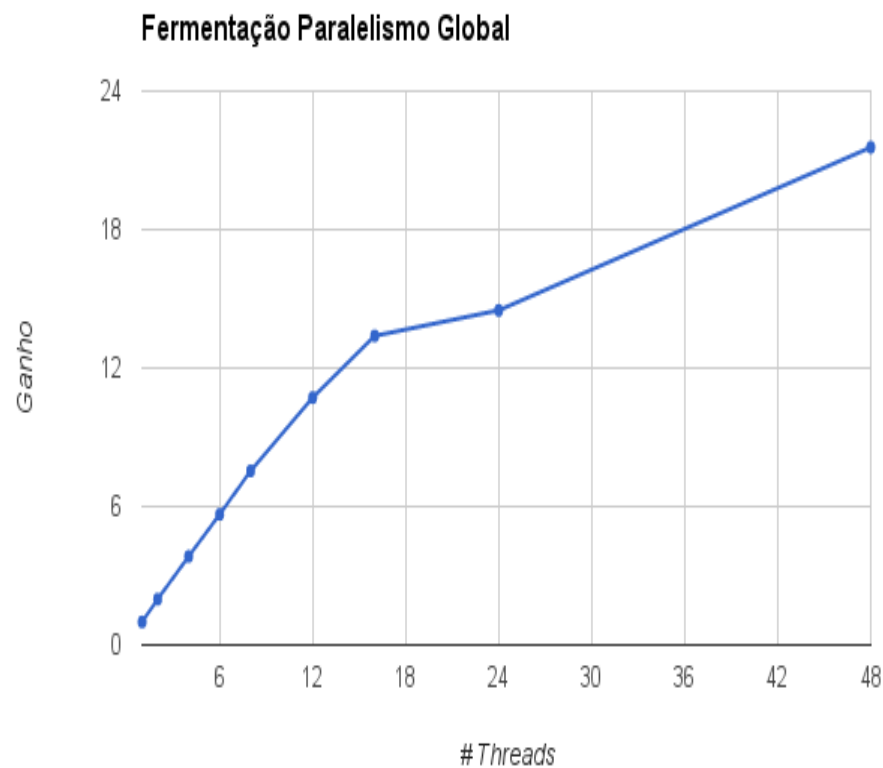


- Speed-up for Knockout and fermentation problem on cluster using the island model



- Solution quality for the fermentations problem
- Migration rate imposes a trade-off:
 - Decreasing migration rate increases speed-up but also decreases the solution quality

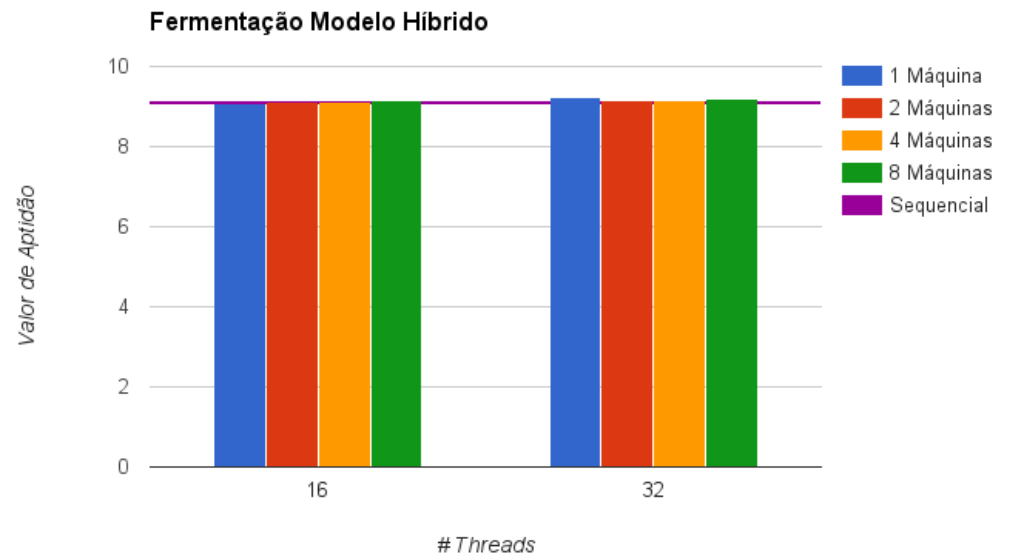
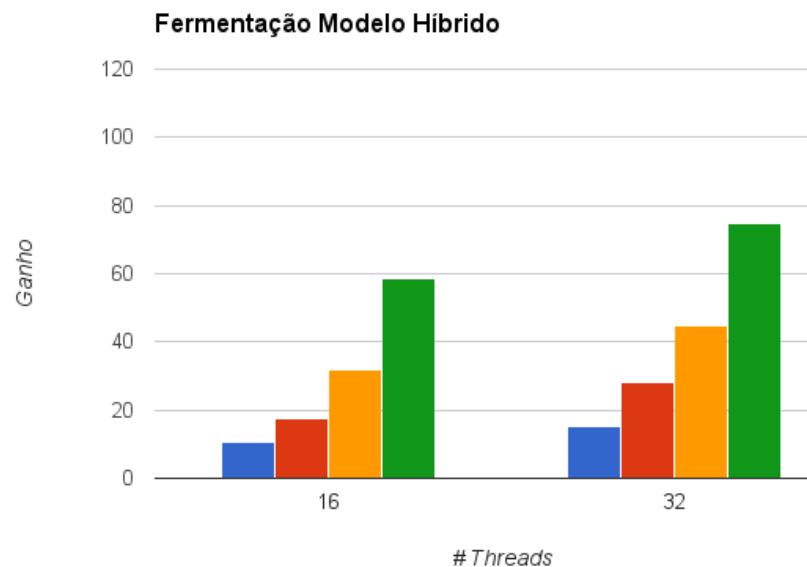
17/19



Results (Tiago)

18/19

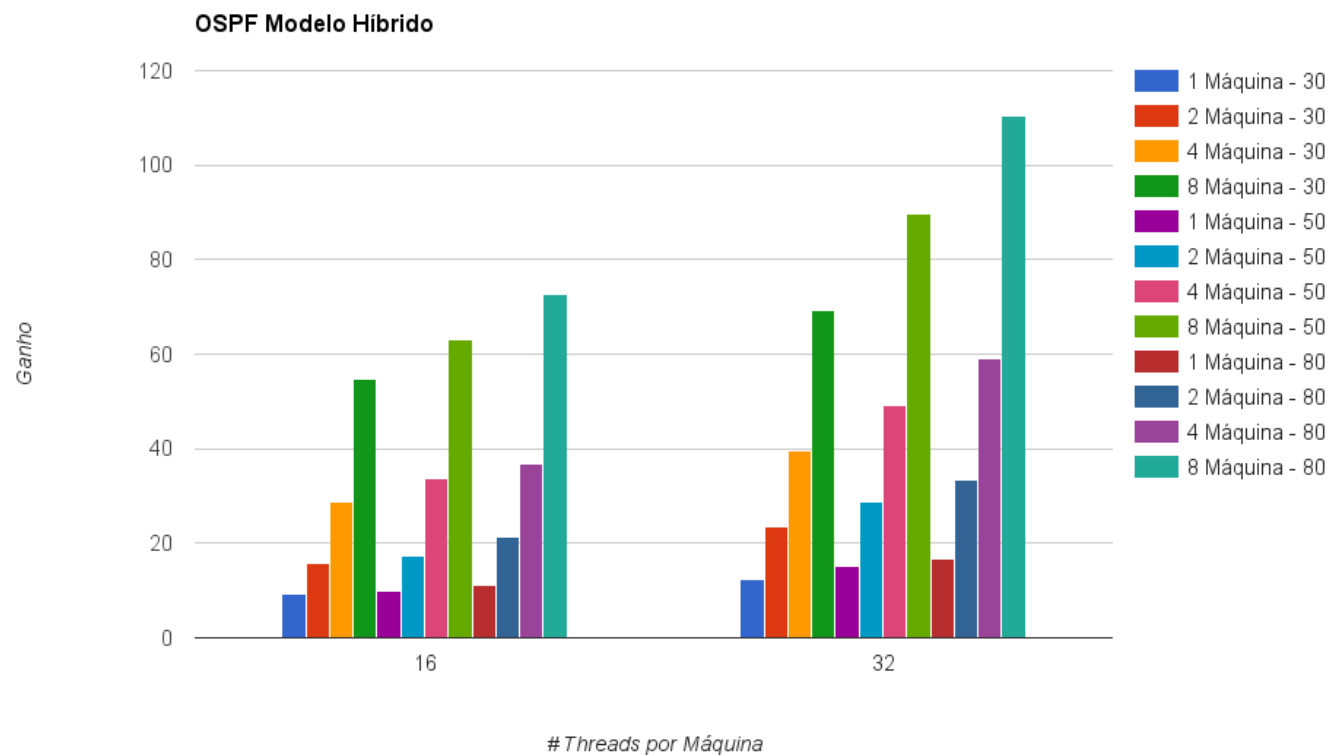
□ Hybrid coarse-grain and global parallelization



Results (Tiago)

19/19

□ Data size impact (OSPF)



Results (Tiago)

20/19

□ Conclusão

Implementação	Vantagens	Desvantagens
<i>Paralelismo Global</i>	+ Configuração e implementação simples; + Paralelismo mais flexível (possui limite no número total de avaliações por geração).	– Implementação em memória partilhada; – Paraleliza apenas uma parte do AE; – Sincronização entre gerações.
<i>Modelo de Ilhas</i>	+ Implementação em memória distribuída; + Paraleliza todas as partes do AE; + Migração assíncrona possibilita mais flexibilidade na comunicação.	– Configuração e implementação complexa; – Paralelismo menos flexível (possui limite no número de indivíduos por ilha).